

Benchmarking local and commercial LLMs for political science text classification

Hanno Hilbig*

2026-05-06

Abstract

I benchmark five local open-weight models against four commercial API models, two from OpenAI plus Anthropic and DeepSeek, on eighteen political science classification tasks. Seventeen tasks use 500 sampled items; the Brandt political-relevance task uses all 320 available cleaned items. On the cross-task average, the strongest API model is Claude Sonnet 4.6 at 0.644 macro F1, while the best local model is Gemma 4 26B at 0.638. Top point estimates are split across six of the nine models, and task-level uncertainty remains substantial. Local inference is slowest on the largest dense architecture, but one local mixture-of-experts model is also the fastest model in the comparison. Batching helps on short prompt tasks with limited accuracy loss at moderate batch sizes, but increases the rate of malformed output on tasks with long, multi-class codebooks. Overall, local open-weight models are competitive on many applied political science classification tasks. However, my results do not support a simple local versus API ranking.

1 Motivation

Political scientists and researchers in adjacent fields who use LLMs for text classification face a practical choice. Commercial API models are reliable, well-documented, and easy to call from R or Python. However, they also cost money per item, send the data off-machine, and create reproducibility risk when the underlying model changes between versions. Local open-weight models running through Ollama or vLLM avoid per-item API charges, keep the data on the researcher’s machine, and make exact model versions easier to preserve.

When it comes to evaluating local model performance, the main question is not “are local models as good as API models in general?” but “is the local model good enough on the specific task I want to run?” In this report, I address four practical sub-questions: are local models competitive with commercial ones on common political science classification tasks? How long do they take, and can they run on a single laptop? Does batching speed them up? And how much does the answer vary across tasks?

Across the eighteen tasks, the top four models (Claude Sonnet 4.6, Gemma 4 26B, Gemma 4 31B, and gpt-5.5) sit within 0.008 macro F1 of each other on the cross-task average, with per-task winners split across six of the nine models. Local inference can match or beat the commercial API tier on speed, and batching helps on short prompt tasks but raises the rate of malformed output on tasks with long, multi-class codebooks.

2 Design

The benchmark covers eighteen classification tasks drawn from published political science replication archives, nine models (five local, four commercial API: two OpenAI, one Anthropic, one DeepSeek), and 320 to 500 items per task sampled with a fixed random seed where sampling is needed. The same items are evaluated by each model. I report macro F1, accuracy, Matthews correlation coefficient (MCC), latency, and the rate of malformed output (the share of model responses that could not be parsed into a valid label). The CSV

*hhilbig@ucdavis.edu. Code and data: <https://github.com/hhilbig/polsci-open-bench>.

column for the latter is `parse_err_rate`. Macro F1 is the main cross-task summary because it is defined for all tasks and gives each class equal weight, but it is not sufficient on its own for imbalanced label sets, where accuracy and MCC are more informative. All comparisons use 95% paired-by-item bootstrap confidence intervals (1000 iterations).

I ran each model on each item once. Local Ollama calls used temperature 0.1. The commercial API models used provider-default decoding, with `reasoning_effort=medium` for the OpenAI tiers. I sampled items with a fixed random seed (20260422), and the sampling is reproducible. Local inference ran on a single Apple Silicon MacBook (32 GB) through Ollama, without the optional internal-reasoning (“thinking”) mode. All five local models were 4-bit quantized variants, so the local results should be read as commodity-hardware inference rather than full-precision performance. OpenAI and DeepSeek calls used JSON-constrained API outputs. Anthropic calls used tool-use to constrain outputs to the task label schema. I use prompts that are verbatim from the source paper’s coding instructions where possible. Where the source paper did not use an LLM, I derive the prompt from the published codebook (per-task provenance is in Table 1).

The serial benchmark includes all nine models. The batched inference comparison remains narrower: batch sizes 10 and 20 are available for six of the original models on the original ten-task grid. Claude Sonnet 4.6 appears only as a serial baseline in that batched summary, and DeepSeek V4 Pro plus Gemma 4 26B are serial-only in the current report. Full-grid batched inference for the expanded 18-task x 9-model benchmark is still outstanding.

3 Results

3.1 Local models are competitive on average

The top four models are close on average. Claude Sonnet 4.6 has the highest mean macro F1 (0.644), followed by Gemma 4 26B (0.638), Gemma 4 31B (0.637), and gpt-5.5 (0.636). The spread across these four is 0.008 points. All nine models fall within roughly 0.06 macro F1 of each other (Figure 1), while per-task spread within any single model exceeds that range. Most task-level model differences fall inside bootstrap CI overlap. The conclusion is robust to the metric choice: the same top group sits at the top under mean accuracy and mean MCC, although the exact order changes slightly. Among the OpenAI tiers, gpt-5.4-nano remains the cost-efficient option, now at 0.620 mean macro F1 compared with 0.636 for gpt-5.5.

The larger differences are across tasks. At 320 to 500 items per task, the bootstrap intervals are wide enough that many task-level leader differences remain uncertain. Task-specific validation matters more than small differences between highly ranked models.

3.2 No model wins everywhere

Figure 2 aggregates the eighteen tasks into five families. The family-level averages do not show a simple local versus API pattern. gpt-5.5 remains strongest on several high-performing policy and event tasks, Claude Sonnet 4.6 leads the cross-task average, and Gemma 4 26B/31B are close to the API leaders overall. At the task level, the top point estimates are split across six models: gpt-5.5 leads on nine tasks, Claude Sonnet 4.6 on three, Qwen3-14B and DeepSeek V4 Pro on two each, and Gemma 4 26B plus gpt-5.4-nano on one each.

3.3 Local inference is not uniformly slower

Local inference speed varies sharply with model size and architecture (Figure 3). The fastest model is Qwen3-30B-A3B (a sparse “mixture-of-experts” model that uses about three billion active parameters per call), at about 0.6 s/item. Gemma 4 26B is only slightly slower on median latency and has the strongest local cross-task F1. The slowest local model is Gemma 4 31B, the largest dense local model, at about 4.4 s/item on the median task. For applied use, parameter count alone is a poor guide to speed: the sparse local models are faster than several smaller dense alternatives.

All five local models ran the full benchmark on a single 32 GB Apple Silicon MacBook; the largest, Gemma 4 31B, fit comfortably in unified memory. Latencies will reorder on different hardware, especially among

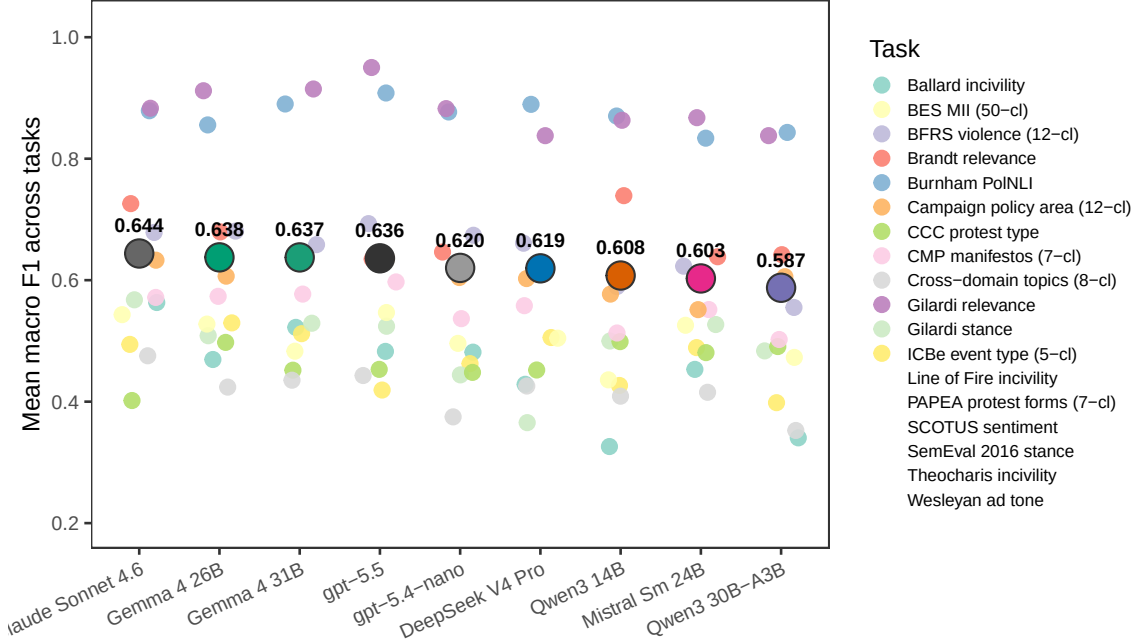


Figure 1: Mean macro F1 across eighteen tasks per model. Large filled circles show cross-task means (numeric labels); small dots show per-task macro F1 values, coloured by task.

the local models, depending on memory and compute. For reference, the API tiers in this run sat between roughly 0.7 and 1.7 s/item on median latency end-to-end, but those numbers mix model speed with provider serving load, network, and queueing, so they are not directly comparable to local inference time and are not shown in the figure.

3.4 Batching helps on short prompts, hurts on long codebooks

Batched prompting sends multiple items in one call and asks the model to return an array of labels, rather than sending one item at a time. It is a common way to reduce per-item latency for API models (Pipal et al. 2026). I tested batch sizes of 10 and 20 against a serial baseline ($b=1$). Batching helps on short prompt tasks with little or no accuracy loss (Figure 4), but creates problems on tasks with long, multi-class codebooks (12 to 50 categories), where the model already has a long instruction prompt to process on every call.

On the short prompt tasks (Gilardi relevance/stance, Ballard incivility, SemEval, SCOTUS), the six models in the batched comparison reach $1.2\times$ to $2.7\times$ per-item speedup at $b=10$. Wesleyan ad tone is the exception in the short prompt set: most local models slow down slightly under batching there. Mean-F1 changes are mostly within ± 0.05 , with a handful of cells (e.g., Wesleyan $\times$ Qwen3-30B-A3B at $b=10$) hitting larger swings. $b=10$ captures most of the speedup, and the move from $b=10$ to $b=20$ buys little extra throughput while raising fragility on smaller local models.

On long codebook tasks (Halterman CCC, BFRS, BES MII, CMP manifestos), two patterns emerge. Local Ollama batching does not help: the instructions are already long, so packing additional items does not save the cost the model pays to read them. A pilot cell (Halterman CCC $\times$ Gemma $\times$ $b=10$) ran at 14.9 s/item, slightly slower than the serial 14.1 s/item baseline. OpenAI batching looks fast in latency terms, but the rate of malformed responses climbs sharply. Figure 4 understates this problem because F1 is computed only on outputs that can be parsed. Failed responses are a separate reliability cost. gpt-5.5 batched on Halterman CCC at $b=10$ returns malformed output for 68% of items, and gpt-5.5 on Wesleyan at $b=20$ hits 48%. gpt-5.4-nano tops out at 12% on the same tasks and is more reliable under batching.

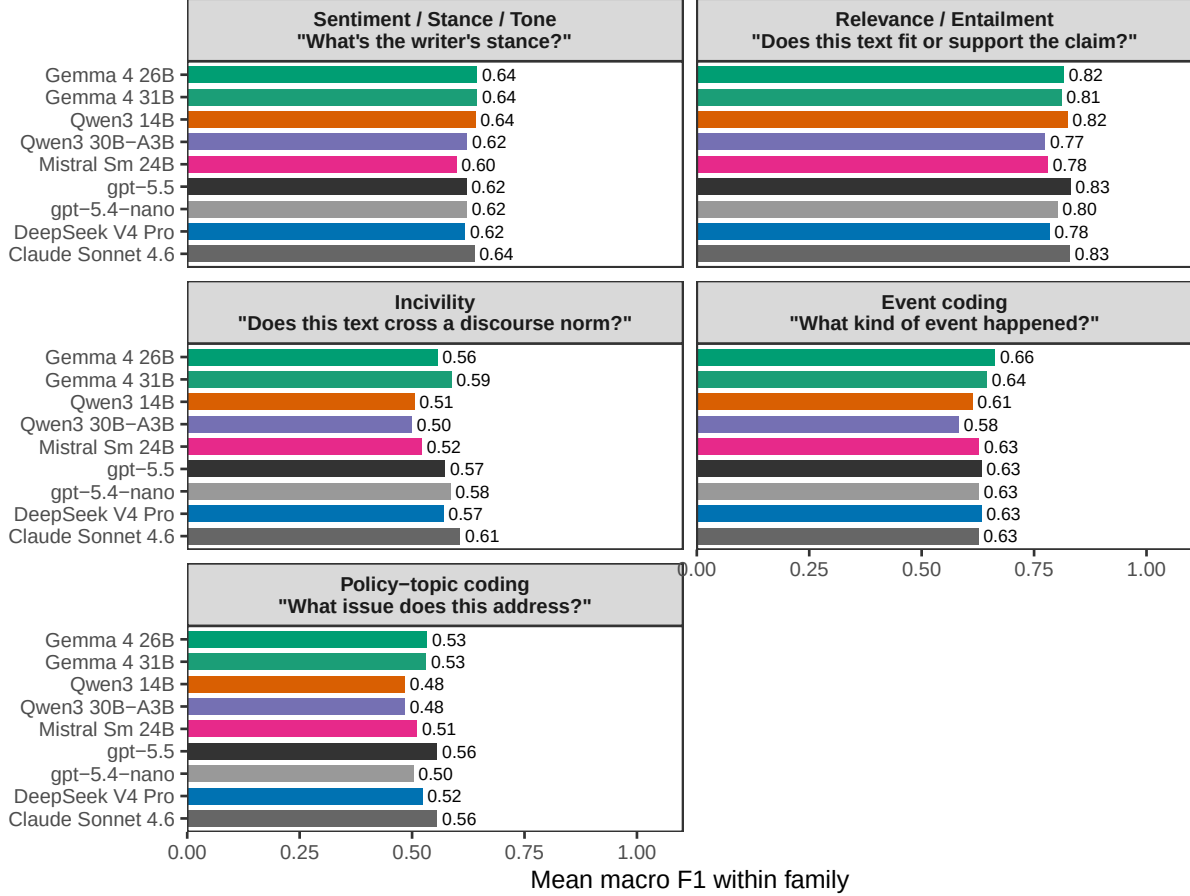


Figure 2: Mean macro F1 within task family, by model. Each family contains three to four tasks; family-level averages are descriptive only.

3.5 Cost

The OpenAI portion of the expanded serial run remains cheap enough for validation-scale benchmarking but differs sharply by tier. In the original 10-task grid, gpt-5.4-nano cost about \$1.20 for 5,000 predictions, compared with \$21.66 for gpt-5.5. In the expanded 18-task results, gpt-5.4-nano trails gpt-5.5 by 0.016 mean macro F1 (0.620 vs 0.636). Claude Sonnet 4.6 is the top model in the expanded grid, but its token-priced cost is closer to the flagship tier than to nano. Local models avoid API charges, but the relevant cost becomes wall-clock time, hardware availability, and energy use.

Implications for applied use

The results imply a workflow rather than a single best model. For researchers, a small validation sample on the target task should come before production use. At this scale, the benchmark average does not reliably predict which model performs best on any specific task, and task-level reversals are common. All five local models fit on a 32 GB Apple Silicon MacBook (the largest dense model takes about 4.4 seconds on the median task, the fastest about 0.6), so a one-laptop setup is enough hardware for the full benchmark.

For metric reporting on imbalanced label sets, accuracy and MCC should appear alongside macro F1. Mellon BES MII is a clean illustration: for the top model, macro F1 is about 0.55, accuracy is 0.95, and MCC is 0.93.

For batching, $b=10$ is a reasonable starting point on short prompt tasks. The accuracy cost is small in this benchmark and the speedup is meaningful. For tasks with long, multi-class codebooks, my results suggest

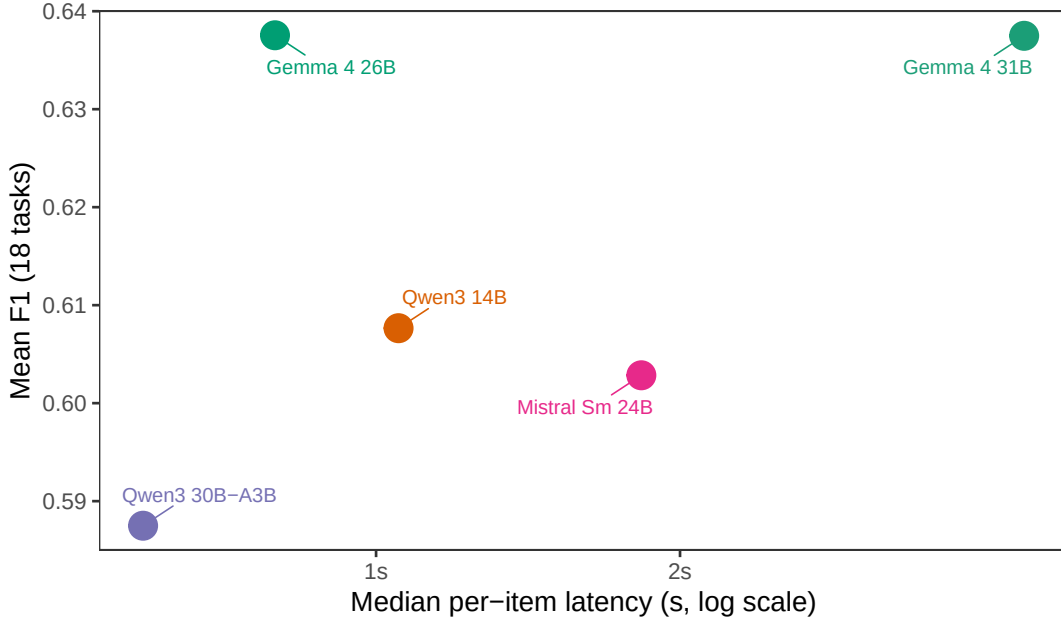


Figure 3: Mean F1 across tasks vs median per-item latency, for the five local models. API models are excluded because their per-call wall-clock mixes server load, queueing, and network round-trip and is not directly comparable to local inference time.

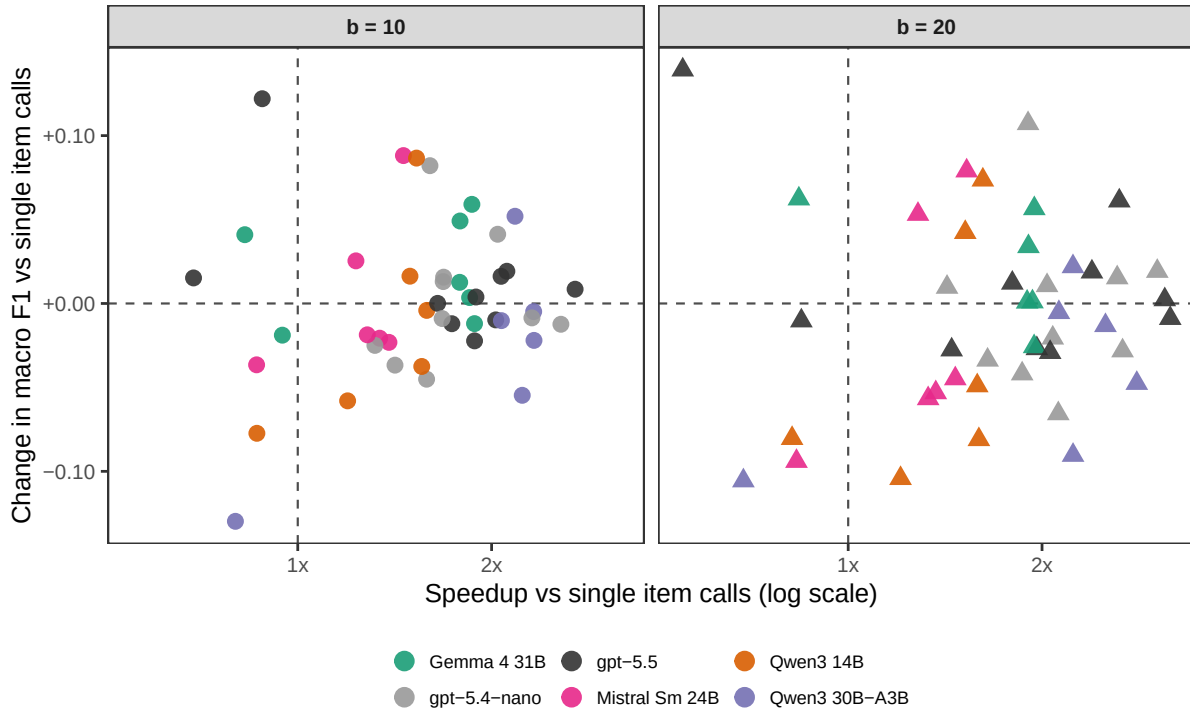


Figure 4: Per-cell speedup vs accuracy change for batched inference relative to single item (serial) calls, faceted by batch size ($b=10$ left, $b=20$ right). Each point is one (task, model) cell. Upper-right = faster and at-least-as-accurate, lower-right = fast but worse, left of $x=1$ = batching does not help here. Malformed output rates are not shown in this figure. They are discussed in the text.

researchers should not batch aggressively unless retries are built in. Malformed output rates on gpt-5.5 reach 26 to 68 percent at b=10/20 on Halterman CCC, BFRS, and Wesleyan, and gpt-5.4-nano is the more reliable batched option for that workload.

Limitations and scope

Model selection and prompting. The model set reflects a practical applied setup, not the full set of available models: five local models that fit a 32 GB Apple Silicon setup, plus four commercial API tiers that include cheap and flagship options. I did not optimize prompts separately per model. Some are verbatim from the source paper, others are derived from published codebooks. This makes the benchmark closer to typical applied use, but it is not a maximum-performance comparison.

Task averaging and rare labels. The mean F1 ranking is descriptive. It is not usually the target quantity for an applied coding project. Tasks differ in label count, class imbalance, construct complexity, and prompt provenance. For imbalanced codebooks, average metrics can hide failures on rare classes.

Beyond this benchmark. Local inference keeps text on the researcher’s machine, which can simplify workflows with sensitive or restricted data. The benchmark studies prompt based classification only. For large labeled datasets and classification tasks with a fixed label set, fine-tuned local models, supervised classifiers, or embedding-based methods may be cheaper and more reliable. API endpoints can change behavior between releases even when the model name is stable, while local open-weight models are easier to freeze exactly.

References

- Ballard, A. O., DeTamble, R., Dorsey, S., Heseltine, M., & Johnson, M. (2022). Incivility in Congressional tweets. *American Politics Research*.
- Brandt et al. (2025). Extractive versus generative language models for political conflict text classification. *Political Analysis*.
- Burnham, Kahn, Wang, and Peng. (2025). Political DEBATE / PolNLI.
- Chae, Y., & Davidson, T. (2025). Large language models for text classification: From zero-shot learning to instruction-tuning. *Sociological Methods & Research*.
- Douglass et al. (2024). ICBs sentence-level event annotations.
- Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30).
- Halterman, A., & Keith, K. A. (2025). Codebook LLMs: Evaluating LLMs as measurement tools for political science concepts. *Political Analysis*.
- Haunss et al. (2025). PAPEA: A modular pipeline for the automation of protest event analysis. *Political Science Research and Methods*.
- Mellon, J., Bailey, J., Scott, R., Breckwoldt, J., Miori, M., & Schmedeman, P. (2024). Do AIs know what the most important issue is? Using language models to code open-text social survey responses at scale. *Research & Politics*.
- Muller, and Fujimura. (2024). Campaign communication and policy-area coding.
- Ornstein, J. T., Blasingame, E. N., & Truscott, J. (2025). How to train your stochastic parrot: Large language models for political texts. *Political Science Research and Methods*.
- Osnabruegge, Ash, and Morelli. (2023). Cross-domain topic classification.
- Pipal, M., Vogel, S. J., Wack, M., & Esser, F. (2026). Researchers waste 80% of LLM annotation costs by classifying one text at a time. *arXiv:2604.03684*.

Rheault, Rayment, and Musulan. (2019). Line-of-fire incivility annotations. *Research & Politics*.

Theocharis et al. (2020). Dynamics of political incivility. *Sage Open*.

Zhang, M., Cakmak, F., Neumann, M., Zimmeck, S., Oleinikov, P., Yao, J., Yu, H., Jacewicz, A., Tassone, I., Floyd, B., Baum, L., Franz, M. M., Ridout, T. N., & Fowler, E. F. (2025). Comparable 2022 general election advertising datasets from Meta and Google. *Scientific Data*.

Companion materials

Prompt provenance, batched inference details, summary CSVs, and reproduction instructions are on the GitHub repository at <https://github.com/hhilbig/polsci-open-bench>.

Tasks

Table 1: Tasks in the benchmark. 'Family' is the grouping used in the family analysis. 'Provenance': Verbatim = prompt unchanged from source paper, Verbatim-adapted = source content with format-only changes, Derived = our prompt reasoned from the published codebook. 'Prompt': short (15-35 lines) or long codebook (47-126 lines). This is the trait that determines local batching feasibility. The CMP task uses the Halterman & Keith (2025) domain-collapsed 7-class version of the CMP/MARPOR coding scheme, not the full CMP category set.

Task	Description	Source	Family	Labels	Provenance	Prompt
Gilardi relevance	Tweets, classified as about content moderation or not.	Gilardi 2023	Relevance / Entailment	binary	Verbatim	short
Brandt relevance	News articles, classified as politics-relevant or not.	Brandt et al. 2025	Relevance / Entailment	binary	Derived	short
Burnham PolNLI	Premise-hypothesis pairs, classified for entailment.	Burnham et al. 2025	Relevance / Entailment	binary	Derived	short
Ballard incivility	Tweets by U.S. members of Congress, classified for incivility.	Ballard 2022	Incivility	binary	Derived	short
Line of Fire incivility	Political tweets, classified for incivility.	Rheault et al. 2019	Incivility	binary	Derived	short
Theocharis incivility	Political tweets, classified for incivility.	Theocharis et al. 2020	Incivility	binary	Derived	short
Gilardi stance	Tweets about content moderation, classified by stance toward moderation.	Gilardi 2023	Sentiment / Stance / Tone	3-class	Verbatim	short
SemEval stance	Tweets toward five named political targets, classified by stance.	Chae & Davidson 2025	Sentiment / Stance / Tone	3-class	Derived	short
SCOTUS sentiment	Tweets about U.S. Supreme Court rulings, classified by sentiment.	Ornstein et al. 2025	Sentiment / Stance / Tone	3-class	Derived	short
Wesleyan ad tone	U.S. Meta political ads, classified by tone toward a candidate.	Zhang et al. 2025	Sentiment / Stance / Tone	3-class	Derived	short
CCC protest type	News stories about U.S. protest events, classified by event type.	Halterman & Keith 2025	Event coding	4-class	Verbatim-adapted	long codebook
BFRS violence	News stories about Pakistani political violence, classified by event type.	Halterman & Keith 2025	Event coding	12-class	Verbatim-adapted	long codebook
PAPEA protest forms	Protest snippets, classified by protest form.	Haunss et al. 2025	Event coding	7-class	Derived	long codebook

ICBe event type	Crisis-event sentences, classified by sentence event type.	Douglass et al. 2024	Event coding	5-class	Derived	short
CMP manifestos	Quasi-sentences from political party manifestos, classified by policy domain.	Halterman & Keith 2025	Policy-topic coding	7-class	Derived	long codebook
Cross-domain topics	Political text, classified by broad policy domain.	Osnabruegge et al. 2023	Policy-topic coding	8-class	Derived	long codebook
Campaign policy area	Campaign statements, classified by policy area.	Muller & Fujimura 2024	Policy-topic coding	12-class	Derived	long codebook
BES MII	Open-ended British Election Study responses, classified by issue topic.	Mellon et al. 2024	Policy-topic coding	50-class	Verbatim-adapted	long codebook

Per-task winners

Table 2: Top model per task by macro F1, with accuracy and MCC for the same model on the same task. CIs and the full per-(task, model) table are on the GitHub repository.

Task	Top model	Macro F1	Accuracy	MCC
BES MII (50-cl)	gpt-5.5	0.547	0.946	0.934
BFRS violence (12-cl)	gpt-5.5	0.693	0.730	0.691
Ballard incivility	Claude Sonnet 4.6	0.563	0.848	0.515
Brandt relevance	Qwen3 14B	0.739	0.887	0.703
Burnham PolNLI	gpt-5.5	0.908	0.928	0.853
CCC protest type	Qwen3 14B	0.499	0.610	0.387
CMP manifestos (7-cl)	gpt-5.5	0.597	0.646	0.545
Campaign policy area (12-cl)	gpt-5.5	0.636	0.750	0.691
Cross-domain topics (8-cl)	Claude Sonnet 4.6	0.476	0.494	0.433
Gilardi relevance	gpt-5.5	0.950	0.954	0.908
Gilardi stance	Claude Sonnet 4.6	0.568	0.648	0.471
ICBe event type (5-cl)	Gemma 4 26B	0.530	0.584	0.463
Line of Fire incivility	DeepSeek V4 Pro	0.607	0.886	0.561
PAPEA protest forms (7-cl)	gpt-5.5	0.965	0.972	0.958
SCOTUS sentiment	DeepSeek V4 Pro	0.668	0.718	0.543
SemEval 2016 stance	gpt-5.5	0.773	0.768	0.659
Theocharis incivility	gpt-5.5	0.682	0.812	0.551
Wesleyan ad tone	gpt-5.4-nano	0.711	0.918	0.808

Batched malformed output rates on long codebook tasks

Table 3: Malformed output rates for the two OpenAI tiers on the long codebook tasks under batched inference. Local Ollama models and short prompt tasks are excluded. Rates there are mostly low and discussed in Section 3.4.

Task	Model	Batch size	Malformed output rate
BES MII	gpt-5.4-nano	10	0%
BES MII	gpt-5.4-nano	20	0%
BES MII	gpt-5.5	10	0%
BES MII	gpt-5.5	20	0%
BFRS Pakistan	gpt-5.4-nano	10	12%
BFRS Pakistan	gpt-5.4-nano	20	8%
BFRS Pakistan	gpt-5.5	10	28%
BFRS Pakistan	gpt-5.5	20	32%
CCC protest	gpt-5.4-nano	10	6%
CCC protest	gpt-5.4-nano	20	8%
CCC protest	gpt-5.5	10	68%
CCC protest	gpt-5.5	20	40%
CMP manifestos	gpt-5.4-nano	10	0%
CMP manifestos	gpt-5.4-nano	20	0%
CMP manifestos	gpt-5.5	10	10%
CMP manifestos	gpt-5.5	20	0%
Wesleyan ads	gpt-5.4-nano	10	0%
Wesleyan ads	gpt-5.4-nano	20	0%
Wesleyan ads	gpt-5.5	10	26%

Performance under alternative metrics

Table 4: Cross-task means of macro F1, accuracy, and MCC, by model. The same broad top group appears across all three metrics, while the exact order changes slightly because some imbalanced tasks have high chance accuracy.

Model	Mean macro F1	Mean accuracy	Mean MCC
Claude Sonnet 4.6	0.644	0.750	0.610
Gemma 4 26B	0.638	0.759	0.601
Gemma 4 31B	0.637	0.755	0.609
gpt-5.5	0.636	0.753	0.611
gpt-5.4-nano	0.620	0.740	0.595
DeepSeek V4 Pro	0.619	0.748	0.592
Qwen3 14B	0.608	0.751	0.583
Mistral Sm 24B	0.603	0.726	0.569
Qwen3 30B-A3B	0.587	0.739	0.559